



МОСКОВСКИЙ УНИВЕРСИТЕТ ИМЕНИ С.Ю. ВИТТЕ

Факультет Информационных технологий
Кафедра Кафедра информационных систем
Направление подготовки/Специальность Прикладная информатика

ОТЧЕТ

о прохождении

Производственная практика

/вид практики/

Эксплуатационная практика

/тип практики/

Студента Мухамадиярова Вадима Рафаиловича
(фамилия, имя, отчество)

Место прохождения практики ЧОУВО «МУ им. С.Ю. Витте»

Период прохождения практики с 20.04.2026 г. по 31.05.2026 г.

г. Москва 2026 г.

Содержание

1. Введение и постановка задачи
 2. Теоретические основы
 - 2.1 Детектор границ Canny
 - 2.2 Преобразование Хафа
 - 2.3 Маскирование области интереса (ROI)
 3. Архитектура реализации
 - 3.1 Общая структура программы
 - 3.2 Модуль lanes.py
 - 3.3 Главный модуль main.py
 4. Детальный разбор алгоритма
 - 4.1 Предобработка кадра
 - 4.2 Маска ROI
 - 4.3 Преобразование Хафа — параметры
 - 4.4 Классификация и усреднение линий
 - 4.5 Стабилизация через историю кадров
 - 4.6 Визуализация результата
 5. Тестирование
 - 5.1 Условия тестирования
 - 5.2 Успешные сценарии
 - 5.3 Проблемные сценарии
 6. Анализ ограничений
 7. Сравнение с нейросетевым подходом
 8. Выводы
- Список источников

1. Введение и постановка задачи

Автоматическое детектирование дорожной разметки одна из ключевых задач систем помощи водителю (ADAS, Advanced Driver Assistance Systems) и автономных транспортных средств. Точное определение положения полос позволяет реализовывать такие функции, как предупреждение о выезде за пределы полосы (LDW), удержание в полосе (LKA), а также служит базовым модулем для алгоритмов планирования траектории.

Существуют два принципиально разных подхода к решению этой задачи: классический алгоритмический (на основе геометрии и обработки изображений) и нейросетевой (на основе обучения с учителем). Данная работа посвящена реализации классического подхода — это позволяет понять физический смысл каждого шага обработки, не прибегая к обучению на больших наборах данных.

Классический пайплайн строится на следующей идее: разметка на дороге это линии с характерным контрастом относительно асфальта. Детектор границ Canny выделяет эти контрасты, а преобразование Хафа находит прямолинейные структуры среди обнаруженных границ. Дополнительные шаги фильтрации и усреднения превращают набор разрозненных отрезков в две чёткие линии левую и правую границы полосы.

1.1 Постановка задачи

Требуется разработать программу, которая в реальном времени обрабатывает видеопоток с камеры, установленной в автомобиле, и на каждом кадре:

1. Определяет положение текущей полосы движения.
2. Отображает найденные линии поверх оригинального кадра.
3. Закрашивает область текущей полосы полупрозрачной заливкой.
4. Сохраняет стабильность отображения при временной потере разметки.

Дополнительные требования: алгоритм должен работать без нейросетей, только на основе методов классической обработки изображений; время обработки одного кадра должно быть достаточным для работы в реальном времени (не менее 25 кадров в секунду).

2. Теоретические основы

2.1 Детектор границ Canny

Алгоритм Canny (John F. Canny, 1986) является одним из наиболее широко используемых детекторов границ в компьютерном зрении. Он работает в несколько этапов:

- Сглаживание гауссовым фильтром для подавления шума.

- Вычисление градиента интенсивности по горизонтали и вертикали (операторы Собеля).

- Подавление немаксимумов (non-maximum suppression) оставляет только локальные максимумы градиента вдоль направления градиента.

- Двойная пороговая фильтрация: пиксели с градиентом выше верхнего порога считаются сильными границами, ниже нижнего - шумом, между порогами - слабыми границами.

- Анализ связности: слабые границы сохраняются только если они связаны с сильными.

2.2 Преобразование Хафа

Преобразование Хафа - математический метод обнаружения геометрических примитивов (прямых, окружностей, эллипсов) на изображении. Основная идея заключается в переходе из пространства изображения в пространство параметров.

Каждая точка (x, y) на изображении соответствует синусоиде в пространстве параметров (ρ, θ) . Точки, лежащие на одной прямой, дают синусоиды, пересекающиеся в одной точке пространства параметров. Поиск прямых сводится к поиску точек с максимальным числом пересечений.

Параметрическое уравнение прямой в полярных координатах:

$$\rho = x \cdot \cos(\theta) + y \cdot \sin(\theta)$$

где ρ - расстояние от начала координат до прямой, θ - угол между прямой и осью X. Такая параметризация позволяет представить вертикальные прямые (у которых наклон бесконечен в декартовой системе).

В данной работе используется HoughLinesP (Probabilistic Hough Transform). В отличие от стандартной версии, она анализирует случайное подмножество граничных точек и возвращает не бесконечные прямые, а конечные отрезки. Это снижает вычислительную сложность и даёт более практичный результат.

2.3 Маскирование области интереса

Маска области интереса (Region of Interest, ROI) - стандартный приём снижения числа ложных срабатываний. Дорожная разметка занимает только нижнюю часть кадра; верхняя часть содержит небо, деревья, здания и другие объекты, которые могут ошибочно детектироваться как линии.

Форма маски - трапеция: широкое основание внизу кадра (охватывает всю ширину дороги) сужается к точке схода перспективы вверху. Это соответствует тому, как выглядит дорога с точки зрения камеры - рельсы сходятся к горизонту.

Маска реализуется через создание бинарного изображения (0 - чёрный, 255 - белый) с залитым полигоном, после чего применяется побитовое И (bitwise AND) с результатом детектора Canny.

3. Архитектура реализации

3.1 Общая структура программы

Программа разделена на два модуля:

- `lanes.py` - библиотека алгоритмов обработки изображения: все функции детектирования, фильтрации и визуализации.
- `main.py` - точка входа: захват видео, цикл по кадрам, вызов функций из `lanes.py`, отображение результата.

Такое разделение обеспечивает повторное использование кода - функции из `lanes.py` можно применять к произвольным источникам видео, фотографиям или потоку с камеры без изменения логики.

Схема обработки одного кадра:

`frame` → `canny()` → `mask()` → `HoughLinesP()` → `average_slope_intercept()`
→ `display_lines()` + `fill_lane()` → `addWeighted()` → `imshow()`

3.2 Модуль `lanes.py`

Модуль содержит шесть публичных функций и одну вспомогательную:

Функция	Назначение
<code>canny(img)</code>	Предобработка: <code>grayscale</code> → <code>GaussianBlur</code> → <code>Canny</code>
<code>mask(image)</code>	Применение трапецевидной маски ROI
<code>average_slope_intercept(image, lines)</code>	Классификация и усреднение отрезков Хафа
<code>make_coordinates(image, params)</code>	Вычисление координат итоговой линии
<code>display_lines(image, lines)</code>	Отрисовка линий на чёрном холсте
<code>fill_lane(image, lines)</code>	Полупрозрачная заливка полосы
<code>_build_result(left, right)</code>	Формирование итогового массива линий

3.3 Главный модуль `main.py`

Основной цикл программы построен следующим образом:

```
video = cv.VideoCapture('road.mp4')
while video.isOpened():
    _, frame = video.read()
    # ... обработка кадра ...
    if cv.waitKey(1) & 0xFF == ord('q'): break
```

Обработка каждого кадра обёрнута в блок try/except, что предотвращает аварийное завершение при проблемных кадрах (например, в конце видео или при временной потере разметки). Оригинальный кадр копируется в `copy_img` до любой обработки - это важно, так как функции `canny` и `mask` изменяют изображение на месте, а для наложения результата нужен чистый оригинал.

4. Детальный разбор алгоритма

4.1 Предобработка кадра

Первый шаг - перевод цветного кадра в оттенки серого. Детектор Canny работает с градиентами яркости, цветовая информация для этого не нужна, а работа с одноканальным изображением втрое быстрее:

```
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
```

Затем применяется гауссово размытие с ядром 5×5. Его цель - сгладить высокочастотный шум (случайные яркие и тёмные пиксели), который иначе порождал бы ложные границы:

```
blur = cv.GaussianBlur(gray, (5, 5), 0)
```

Параметр сигма=0 означает автоматический выбор стандартного отклонения по размеру ядра: $\sigma = 0.3 \cdot ((\text{ksize}-1) \cdot 0.5 - 1) + 0.8 \approx 1.1$. Ядро 5×5 - стандартный выбор; ядро 3×3 оставляет больше шума, а 7×7 и более размывает мелкие детали разметки.

Завершает предобработку детектор Canny:

```
return cv.Canny(blur, 50, 150)
```

Нижний порог 50 означает, что пиксели с градиентом ниже 50 полностью отбрасываются. Верхний порог 150 - пиксели с градиентом выше 150 считаются надёжными границами. Пиксели между 50 и 150 сохраняются только если соединены с надёжными границами. Соотношение порогов 1:3 - рекомендация автора алгоритма.

4.2 Маска ROI

Маска задаётся четырьмя точками трапеции в относительных координатах:

```
polygons = np.array([
    (int(width * 0.10), height),          # нижний левый
    (int(width * 0.45), int(height * 0.60)), # верхний левый
    (int(width * 0.55), int(height * 0.60)), # верхний правый
    (int(width * 0.90), height),          # нижний правый])
```

Верхняя граница маски установлена на уровне 60% высоты кадра. Это примерно соответствует горизонту при типичной установке камеры на капоте автомобиля. Нижняя граница начинается на 10% от края кадра с каждой стороны - это отсекает боковые объекты (бордюры, обочины), которые не являются разметкой полосы.

Ключевой момент реализации: используется переменная `mask`, совпадающая по имени с функцией. В финальной версии это исправлено - внутренняя переменная переименована в `blank` во избежание конфликта имён.

4.3 Преобразование Хафа — параметры

Функция `HoughLinesP` принимает следующие параметры:

Параметр	Значение	Пояснение
<code>rho</code>	1	Точность по расстоянию — 1 пиксель
<code>theta</code>	$\pi/180$	Точность по углу — 1 градус
<code>threshold</code>	50	Минимум голосов для регистрации линии
<code>minLineLength</code>	30	Минимальная длина отрезка в пикселях
<code>maxLineGap</code>	150	Максимальный пробел внутри отрезка

Параметр `maxLineGap=150` особенно важен для пунктирной разметки: он позволяет соединять отдельные штрихи в один непрерывный отрезок. Без этого пунктир детектируется как множество коротких отрезков, что затрудняет усреднение. Значение 150 подобрано под конкретное видео - на дорогах с более редким пунктиром может потребоваться увеличение.

4.4 Классификация и усреднение линий

Функция `average_slope_intercept` - ядро алгоритма. Для каждого найденного отрезка вычисляются параметры прямой через `numpy`:

```
parameters = np.polyfit((x1, x2), (y1, y2), 1)
slope      = parameters[0] # наклон
intercept  = parameters[1] # точка пересечения с осью Y
```

Классификация по трём критериям:

Фильтр наклона: $|\text{slope}| < 0.4$ - почти горизонтальная линия (не полоса), $|\text{slope}| > 3.5$ - почти вертикальная (артефакт). Обе отбрасываются.

Знак наклона: в системе координат изображения (Y направлена вниз) $\text{slope} < 0$ означает линию, идущую снизу-вправо вверх-влево - это левая полоса.

Положение центра: $\text{mid_x} < \text{center_x}$ для левой полосы и $\text{mid_x} > \text{center_x}$ для правой исключает ошибочную классификацию линий, пересекающих центр кадра.

Усреднение параметров внутри группы:

```
left_avg = np.average(left_fit, axis=0) # средний (slope, intercept)
left_line = make_coordinates(image, left_avg)
```

Функция `make_coordinates` восстанавливает координаты отрезка из параметров прямой. Нижняя точка фиксируется у нижнего края кадра ($y1 = \text{height}$), верхняя - на уровне 3/5 высоты ($y2 = \text{height} * 3/5$):

```
x1 = int((y1 - intercept) / slope) # из  $y = \text{slope} \cdot x + \text{intercept}$   
x2 = int((y2 - intercept) / slope)
```

4.5 Стабилизация через историю кадров

Стандартный алгоритм Хафа даёт нестабильные результаты: линии «прыгают» от кадра к кадру, а при временной потере разметки (тень, стык асфальта) пропадают полностью. Для решения этой проблемы реализован механизм истории:

```
_left_history = deque(maxlen=50)
_right_history = deque(maxlen=50)
```

При успешном обнаружении линии она добавляется в историю:

```
_left_history.append(left_line)
```

При отсутствии линии в текущем кадре берётся среднее из истории:

```
left_line = np.mean(_left_history, axis=0).astype(int)
```

Размер очереди 50 кадров - компромисс между двумя крайностями. При меньшем размере (10–15 кадров) история недостаточно сглаживает колебания. При большем (100+ кадров) линия реагирует на изменения слишком медленно - например, при резкой смене полосы она продолжает показывать старое положение ещё несколько секунд.

deque (двусвязная очередь) из модуля collections - оптимальная структура данных для хранения скользящего окна: добавление нового элемента и удаление старого выполняются за $O(1)$.

4.6 Визуализация результата

Визуализация строится в два слоя, которые накладываются на оригинальный кадр:

Слой 1 - заливка полосы (fill_lane):

Четыре точки трапеции (нижняя и верхняя точки каждой линии) образуют полигон, который заливается зелёным цветом на копии оригинального кадра:

```
overlay = image.copy()
cv.fillPoly(overlay, [pts], (0, 255, 0))
return cv.addWeighted(overlay, 0.25, image, 0.75, 0)
```

Коэффициент 0.25 для overlay даёт прозрачность 25% - зелёный цвет виден, но дорога под ним тоже читается.

Слой 2 - линии (display_lines):

Линии рисуются на чёрном холсте (zeros_like), а не на оригинале. Затем холст накладывается через addWeighted:

```
line_image = np.zeros_like(image)
cv.line(line_image, (x1,y1), (x2,y2), (255,255,255), 10)
combo = cv.addWeighted(filled, 0.8, line_image, 0.5, 1)
```

Белый цвет (255, 255, 255) и толщина 10 пикселей обеспечивают хорошую видимость линий на любом фоне.

5. Тестирование

5.1 Условия тестирования

Тестирование проводилось на видеозаписях с видеорегистратора. Характеристики тестовых видео:

Параметр	Значение	Примечание
Разрешение	1280×720	HD, стандарт видеорегистраторов
Частота кадров	30 fps	Стандартная частота
Освещение	Дневной свет	Без прямого солнца в объектив
Тип дороги	Двухполосное шоссе	С чёткой разметкой
Положение камеры	На капоте	Горизонтальный угол
Длительность	~30 секунд	~900 кадров

Производительность алгоритма: среднее время обработки одного кадра составило 6-8 мс на процессоре Ryzen 5 без использования GPU. Это обеспечивает обработку 120+ кадров в секунду - значительно выше требуемых 30 fps.

5.2 Успешные сценарии

Прямая дорога с чёткой разметкой:

На прямых участках с хорошо видимой разметкой алгоритм стабильно детектирует обе линии. Линии не «прыгают» благодаря механизму истории. Зелёная заливка точно соответствует полосе движения.

Пунктирная разметка:

Параметр `maxLineGap=150` успешно объединяет отдельные штрихи пунктира в одну линию. Даже когда большая часть пунктирного штриха оказывается вне маски ROI, алгоритм восстанавливает линию из истории.

Переменная освещённость:

Небольшие тени от деревьев не нарушают работу алгоритма - фильтр наклона отсекает горизонтальные тени, а история сглаживает кратковременные потери линии.

5.3 Проблемные сценарии

Резкий поворот:

На поворотах реальная полоса изгибается. Линия либо смещается к центру поворота, либо пропадает, если изогнутая разметка не вписывается в трапецию

маски ROI. История частично сглаживает пропадание, но положение линии остаётся неточным.

Перекрёсток:

На перекрёстках разметка горизонтальная (пешеходный переход, стоп-линия) или отсутствует. Горизонтальные линии отфильтровываются фильтром наклона, алгоритм опирается на историю предыдущих кадров.

Блики и яркое солнце:

Прямое солнечное освещение создаёт зоны с очень высокой яркостью. Детектор Canny с фиксированными порогами плохо работает в таких условиях - либо не находит границ в пересвеченной зоне, либо находит слишком много ложных границ.

Грязный или мокрый асфальт:

Мокрый асфальт создаёт зеркальные блики и снижает контраст разметки. Грязь покрывает разметку, уменьшая её яркость. В обоих случаях снижается число голосов за линии в пространстве Хафа, что приводит к пропаданию детекции.

6. Анализ ограничений

6.1 Прямолинейное приближение

Фундаментальное ограничение алгоритма представление каждой полосы одной прямой линией. Математически это означает, что алгоритм ищет параметры прямой $y = kx + b$, наилучшим образом описывающей набор точек разметки. На прямой дороге такое приближение работает отлично. На повороте реальная полоса описывается кривой, и прямолинейное приближение даёт систематическую ошибку.

Для кривых дорог существуют альтернативные подходы: параболическое приближение второго порядка ($y = ax^2 + bx + c$), сплайны (кусочно-полиномиальные кривые), а также поиск в перспективно-искажённом виде («вид сверху» bird's eye view), где полосы выглядят параллельными прямыми даже на поворотах.

6.2 Зависимость от параметров

Алгоритм содержит около 10 числовых параметров (пороги Canny, параметры Хафа, координаты маски, фильтр наклона), каждый из которых подобран под конкретное видео. При смене камеры, угла установки, типа дороги или условий освещения параметры необходимо перенастраивать. Нейросетевые подходы лишены этого недостатка: они адаптируются к условиям автоматически за счёт обучения.

6.3 Отсутствие понимания контекста

Алгоритм обрабатывает каждый кадр независимо (за исключением истории). Он не понимает, что линии должны быть параллельны, что ширина полосы постоянна, что на перекрёстке разметка прерывается намеренно. Нейросетевые модели, обученные на тысячах примеров, неявно усваивают эти закономерности.

6.4 Вычислительная неэффективность при увеличении числа линий

В текущей реализации алгоритм ищет ровно две линии - левую и правую. Добавление третьей линии (например, для выделения встречной полосы) требует существенной переработки логики классификации: простое деление по знаку наклона перестаёт работать, если в кадре видны три и более линий с одинаковым знаком.

7. Сравнение с нейросетевым подходом

Для полноты картины сравним реализованный классический алгоритм с современными нейросетевыми подходами к детектированию полос:

Критерий	Классический (Хаф)	Нейросетевой
Прямая дорога	Отлично	Отлично
Повороты	Плохо (прямая линия)	Хорошо (кривые полосы)
Ночные условия	Плохо	Хорошо (если обучена)
Требует обучения	Нет	Да (тысячи размеченных кадров)
Настройка под условия	Ручная (параметры)	Автоматическая (обучение)
Скорость работы	Очень высокая (6-8 мс)	Высокая (10-30 мс на GPU)
Интерпретируемость	Полная	Ограниченная (чёрный ящик)
Требования к железу	CPU достаточно	Обычно нужен GPU
Сложность реализации	Низкая	Высокая

Среди нейросетевых подходов к детектированию полос наиболее распространены:

- Семантическая сегментация - каждому пикселю присваивается класс (полоса / не полоса). Архитектуры: U-Net, DeepLab, SegNet.
- Предсказание опорных точек - сеть предсказывает набор точек, лежащих на полосе, которые затем соединяются сплайном. Архитектуры: LaneNet, Ultra-Fast Lane Detection.
- Параметрическое описание - сеть напрямую предсказывает коэффициенты полинома, описывающего полосу.

Классический подход оправдан в задачах с жёсткими требованиями к интерпретируемости и предсказуемости поведения, при ограниченных вычислительных ресурсах, а также как учебный инструмент для понимания базовых принципов.

8. Выводы

При выполнении практической работы был реализован полный пайплайн детектирования дорожной разметки на основе преобразования Хафа. Программа обрабатывает видеопоток в реальном времени и корректно выделяет левую и правую границы полосы движения в штатных условиях.

Реализованы и изучены следующие компоненты алгоритма:

- Предобработка изображения: перевод в grayscale, гауссово размытие, детектор границ Canny с подобранными порогами 50/150.

- Маскирование ROI: трапециевидный полигон в относительных координатах, исключающий небо и обочины из области поиска.

- Вероятностное преобразование Хафа (HoughLinesP) с параметрами, обеспечивающими объединение пунктирной разметки.

- Классификация отрезков по знаку наклона и положению в кадре; двойная фильтрация по наклону.

- Усреднение параметров группы отрезков в одну итоговую линию через `pr.polyfit` и `pr.average`.

- Стабилизация результата через скользящее среднее по истории 50 последних кадров.

- Двухслойная визуализация: полупрозрачная заливка полосы и отрисовка линий через `addWeighted`.

Практически понятно главное ограничение метода: прямолинейное приближение не позволяет корректно описывать изогнутые полосы на поворотах. Для решения этой проблемы в реальных ADAS-системах используются либо параболические кривые, либо нейросетевые модели, предсказывающие полосу как набор точек.

Реализация наглядно демонстрирует, что практический алгоритм компьютерного зрения - это не одна функция, а цепочка из 6-8 этапов, каждый из которых решает конкретную проблему: шум, ложные срабатывания, нестабильность, неточность аппроксимации. Понимание роли каждого этапа необходимо для отладки и адаптации алгоритма к новым условиям.

Список источников

- Canny J. A Computational Approach to Edge Detection // IEEE Transactions on Pattern Analysis and Machine Intelligence. - 1986. - Vol. 8, No. 6. - P. 679-698.
- Hough P.V.C. Method and means for recognizing complex patterns. - US Patent 3069654, 1962.
- Ballard D.H. Generalizing the Hough Transform to Detect Arbitrary Shapes // Pattern Recognition. - 1981. - Vol. 13, No. 2. - P. 111–122.
- Bradski G., Kaehler A. Learning OpenCV 4. - O'Reilly Media, 2019. - 1024 p.
- OpenCV Documentation. HoughLinesP. - URL:
https://docs.opencv.org/4.x/dd/d1a/group__imgproc__feature.html
- OpenCV Documentation. Canny Edge Detector. - URL:
https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html
- Pan X. et al. Ultra Fast Structure-aware Deep Lane Detection // ECCV. - 2020.
- Neven D. et al. Towards End-to-End Lane Detection: an Instance Segmentation Approach // IEEE IV. - 2018.